

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені Тараса Шевченка
ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
Кафедра програмних систем і технологій

Дисципліна
« Комп'ютерна графіка і візуалізація »

Лабораторна робота № 3

Виконав:	Дацишин Володимир	Перевірила:	Жабська Єлизавета Олегівна
Група	ІПЗ-23		
Форма навчання	денна	Оцінка	
Спеціальність	121		
2026			

Тема: розробка програмного забезпечення для обробки зображень: об'єднання зображень, створення водяного знаку та слайд-шоу.

Мета: ознайомитись з методами обробки зображень для об'єднання зображень, додавання водяного знаку та створення слайд-шоу; отримати практичний досвід у розробці інструментів для роботи з графічними даними, налаштування параметрів текстових елементів на зображенні та автоматизації процесу відображення зображень у слайд-шоу; створити програму з інтуїтивно зрозумілим інтерфейсом, який дозволяє виконувати комплексні перетворення графічних даних.

Завдання для виконання:

1. Об'єднання зображень: Програмно реалізувати можливість об'єднувати два зображення горизонтально або вертикально. При цьому користувач повинен мати можливість вибрати напрямок об'єднання, обрати вихідні зображення, а також визначити режим узгодження геометрії (пропорційне підлаштування під спільну висоту/ширину або збереження вихідних розмірів із заповненням полотна).
2. Створення водяного знаку на зображенні: Розробити інструмент для додавання захисного текстового водяного знаку до зображення. Користувач повинен мати можливість вибрати текст водяного знаку, його положення (пресети або координати), рівень прозорості (альфа-канал), розмір шрифту, колір тексту, кут нахилу, контрастну контурну обводку та напівпрозору тінь для запобігання візуальному злиттю з фоном, а також режим суцільного захисного патерну (tiling).
3. Створення слайд-шоу: Створити графічний інтерактивний програвач (GUI на основі бібліотеки Tkinter), який послідовно відтворює зображення, отримані в результаті виконання завдань лабораторних робіт, із регульованою затримкою. Забезпечити керування відтворенням (Play/Pause, Next, Prev, регулювання швидкості, інформаційна панель) та реалізувати можливість експорту слайд-шоу у повноцінний анімований формат GIF із оптимізованою палітрою.

ХІД РОБОТИ

1. Реалізація методів просторового об'єднання зображень

Об'єднання зображень (Image Concatenation / Stacking) є однією з ключових операцій комп'ютерної графіки, що дозволяє створювати комбіновані панорами, колажі, порівняльні макети «до/після» та багатокомпонентні банери. У рамках завдання розроблено універсальну функцію **merge_images(img1_path, img2_path, output_path, direction, resize_mode, align, bg_color, spacing)**, яка підтримує об'єднання як за горизонталлю (пліч-о-пліч), так і за вертикаллю (один під одним).

Математична та алгоритмічна логіка реалізації враховує невідповідність габаритів вхідних зображень:

1. При горизонтальному об'єднанні двох зображень розмірностями ($W1$, $H1$) та ($W2$, $H2$):

- Якщо обрано режим узгодження `resize_mode = 'fit'` та $H1 \neq H2$, друге зображення пропорційно масштабується за коефіцієнтом $k = H1 / H2$, утворюючи нову ширину $W2' = \text{round}(W2 * k)$. Загальна ширина канви становить $W_total = W1 + W2' + \text{spacing}$, а висота $H_total = H1$. Масштабування виконується високоякісним ресемплінгом Lanczos.

- Якщо обрано режим `resize_mode = 'pad'`, зберігаються оригінальні розміри, канва набуває висоти $H_total = \max(H1, H2)$, а вертикальне розміщення меншого кадру регулюється параметром `align` ('top', 'center', 'bottom').

2. При вертикальному об'єднанні:

- У режимі 'fit' ширина другого зображення приводиться до $W1$: $k = W1 / W2$, $H2' = \text{round}(H2 * k)$. Канва отримує розміри $W_total = W1$, $H_total = H1 + H2' + \text{spacing}$.

- У режимі 'pad' канва отримує $W_total = \max(W1, W2)$, $H_total = H1 + H2 + \text{spacing}$, з можливістю горизонтального центрування або притискання до лівого/правого краю.

Для забезпечення коректного рендерингу зображень із прозорістю вставка проводиться у режимі RGBA із передачею альфа-маски, а при збереженні у форматі JPEG автоматично накладається непрозоре біле тло.

2. Реалізація інструментарію створення та захисту водяних знаків

Водяний знак (Watermark) призначений для захисту інтелектуальної власності, маркування авторських прав та брендування графічних матеріалів. Проте простий рендеринг тексту поверх довільного зображення часто призводить до

повної втрати читабельності на строкатих або контрастних ділянках (наприклад, білий текст на білих хмарах).

У розробленій функції **add_watermark(input_path, output_path, text, position, opacity, font_size, font_color, angle, stroke_width, stroke_color, shadow, tile)** впроваджено комплексний підхід до побудови надійного водяного знаку:

- Рівень прозорості (Alpha Blending): значення непрозорості регулюється у діапазоні 0.0..1.0 (або 0..255). Композиція пікселів здійснюється за формулою $C_{out} = C_{mark} * alpha + C_{base} * (1 - alpha)$ через метод `Image.alpha_composite()`.

- Динамічне позиціонування: розрахунок габаритного прямокутника (Bounding Box) тексту виконується методом `draw.textbbox()`. Підтримуються пресети 'center', 'top-left', 'top-right', 'bottom-left', 'bottom-right' із автоматичним відступом `margin`, а також введення точних піксельних координат (x, y).

- Захист від злиття з фоном: реалізовано двокомпонентний контрастний каркас:

- 1) Контурна обводка (Stroke Outline) шириною `stroke_width` із кольором `stroke_color` (наприклад, чорний контур навколо жовтого/білого тексту);

- 2) М'яка напівпрозора тінь (Drop Shadow) із векторним зміщенням (+3, +3 px), що відокремлює символи від фонових граней.

- Просторова орієнтація (Rotation): текст рендериться на локальному прозорому шарі та обертається на довільний кут (наприклад, 35°) з бікубічною інтерполяцією та збереженням повної геометрії (`expand=True`).

- Захисний патерн (Tiling Mode): при активації `tile=True` генерується регулярна діагональна сітка повторюваних штампів, що унеможливорює фрагментарне обрізання зображення злоумисником.

3. Реалізація інтерактивного слайд-шоу та експорту анімації

Згідно з методичними вказівками, для створення локального програвача використано зв'язку Python + Tkinter. Розроблено клас **SlideshowPlayer**, який забезпечує повноцінний користувацький досвід відтворення графічних матеріалів:

- Графічний інтерфейс: сучасний дизайн у темній колірній гамі із верхнім заголовком, центральною областю перегляду та панеллю керування.

- Інтерактивні елементи: кнопки 'Prev', 'Pause / Play', 'Next', інтерактивний повзунок (Scale) налаштування швидкості від 0.5 до 5.0 с, динамічне інформаційне табло з назвою файлу, роздільною здатністю та індексом

поточного слайду (Slide i / N).

- Алгоритм адаптивного вписування (Letterboxing): зображення автоматично масштабуються під розмір вікна зі збереженням оригінальних пропорцій (Aspect Ratio), уникаючи розтягування чи деформації.

- Гарячі клавіші: реалізовано обробку подій клавіатури: стрілки Вліво/Вправо (навігація), Пробіл (пауза/пуск), Escape (вихід).

- Модуль експорту GIF (**create_slideshow_gif**): виконує приведення набору зображень до єдиного розміру полотна, центрування та генерацію оптимізованого анімованого файлу із підтримкою нескінченного циклу (loop=0) та заданого інтервалу кадру.

ВИКОРИСТАНІ ФУНКЦІЇ

Функція / Метод	Призначення в програмі
PIL.Image.open(fp)	Відкриття графічного файлу та завантаження растрових даних у пам'ять.
PIL.Image.new(mode, size, color)	Створення нового порожнього графічного полотна із заданим колірним режимом і тлом.
PIL.Image.resize(size, resample)	Пропорційне або координатне масштабування із використанням фільтра Lanczos.
PIL.Image.paste(im, box, mask)	Накладання одного зображення на інше із передачею альфа-маски для збереження прозорості.
PIL.Image.alpha_composite(im1, im2)	Попіксельне альфа-змішування двох шарів однакової розмірності у просторі RGBA.
PIL.Image.rotate(angle, expand, resample)	Геометричне обертання растрового шару на заданий кут із бікубічною інтерполяцією.
PIL.ImageDraw.Draw(im)	Ініціалізація графічного контексту малювання на растровому полотні.
ImageDraw.text(xy, text, font, fill, ...)	Рендеринг тексту із параметрами заливки, шрифту, товщини обводки та кольору контуру.
ImageDraw.textbbox(xy, text, font, ...)	Обчислення точних піксельних меж (Bounding Box) тексту для прецизійного позиціонування.
PIL.ImageFont.truetype(font, size)	Завантаження векторного TrueType шрифту (Arial, Calibri тощо) із системи.
PIL.ImageTk.PhotoImage(im)	Перетворення об'єкта Pillow у сумісний графічний тип для відмальовки у вікні Tkinter.
tkinter.Tk(), Frame, Label, Button, Scale	Створення головного вікна, панелей, кнопок та повзунка регулювання швидкості.
root.after(delay, callback)	Організація неблокуючого таймера циклічної зміни слайдів у головному циклі подій Tkinter.
numpy.clip(a, a_min, a_max)	Обмеження числових значень пікселів та компонент альфа-каналу у діапазон [0..255].

ЛІСТИНГ КОДУ ПРОГРАМИ З КОМЕНТАРЯМИ

```
from __future__ import annotations
import os
import glob
import time
from typing import Sequence
import numpy as np
from PIL import Image, ImageDraw, ImageFont

# HELPER FUNCTIONS & COLOR UTILITIES
def get_size_kb(filepath: str) -> float:
    """Returns file size in kilobytes."""
    return os.path.getsize(filepath) / 1024.0 if os.path.isfile(filepath) else 0.0

def save_image(img: Image.Image, output_path: str) -> None:
    """
    Saves an image, automatically creating parent directories.
    Handles converting RGBA/transparency to RGB with white background for JPEG/BMP.
    """
    os.makedirs(os.path.dirname(output_path) or ".", exist_ok=True)
    ext = os.path.splitext(output_path)[1].lower()
    save_img = img
    if ext in (".jpg", ".jpeg", ".bmp") and img.mode in ("RGBA", "LA", "P"):
        rgba = img.convert("RGBA")
        bg = Image.new("RGB", rgba.size, (255, 255, 255))
        bg.paste(rgba, mask=rgba.split()[3])
        save_img = bg
    elif ext in (".jpg", ".jpeg") and img.mode != "RGB":
        save_img = img.convert("RGB")
    save_img.save(output_path)
    size_kb = get_size_kb(output_path)
    print(f"Saved: {output_path} ({save_img.width}x{save_img.height}, {size_kb:.2f} KB)")

def parse_color(color_val: str | tuple | list) -> tuple[int, int, int]:
    """Parses color representation (name, hex, tuple, or comma-separated string)
    into RGB tuple."""
    if isinstance(color_val, (tuple, list)) and len(color_val) >= 3:
        return (int(color_val[0]), int(color_val[1]), int(color_val[2]))
```

```

s = str(color_val).strip().lower()
palette = {
    "white": (255, 255, 255), "black": (0, 0, 0), "red": (255, 0, 0),
    "green": (0, 255, 0), "blue": (0, 0, 255), "yellow": (255, 255, 0),
    "cyan": (0, 255, 255), "magenta": (255, 0, 255), "gray": (128, 128, 128),
    "lightgray": (211, 211, 211), "darkgray": (64, 64, 64),
    "orange": (255, 165, 0), "gold": (255, 215, 0), "navy": (0, 0, 128)
}
if s in palette:
    return palette[s]
if s.startswith("#"):
    s = s[1:]
if len(s) == 6 and all(c in "0123456789abcdef" for c in s):
    return (int(s[0:2], 16), int(s[2:4], 16), int(s[4:6], 16))
if len(s) == 3 and all(c in "0123456789abcdef" for c in s):
    return (int(s[0] * 2, 16), int(s[1] * 2, 16), int(s[2] * 2, 16))

parts = [int(p.strip()) for p in s.split(",") if p.strip()]
if len(parts) >= 3 and all(0 <= p <= 255 for p in parts[:3]):
    return (parts[0], parts[1], parts[2])
return (255, 255, 255)

def load_font(font_size: int, font_name: str | None = None) ->
ImageFont.FreeTypeFont | ImageFont.ImageFont:
    """Loads a TrueType font with graceful fallback to system fonts or default
    bitmap font."""
    candidate_paths = []
    if font_name:
        candidate_paths.append(font_name)
    # Common Windows system fonts
    win_fonts = [
        "arial.ttf", "calibri.ttf", "tahoma.ttf", "segoeui.ttf", "times.ttf",
        "C:/Windows/Fonts/arial.ttf", "C:/Windows/Fonts/calibri.ttf",
        "C:/Windows/Fonts/times.ttf"
    ]
    candidate_paths.extend(win_fonts)

    for path in candidate_paths:
        try:
            return ImageFont.truetype(path, font_size)
        except Exception:
            continue
    try:

```

```

        return ImageFont.load_default(size=font_size)
    except Exception:
        return ImageFont.load_default()

# TASK 1: IMAGE MERGING (ГОРИЗОНТАЛЬНЕ ТА ВЕРТИКАЛЬНЕ ОБ'ЄДНАННЯ)
def merge_images(
    img1_path: str,
    img2_path: str,
    output_path: str,
    direction: str = "horizontal",
    resize_mode: str = "fit",
    align: str = "center",
    bg_color: tuple = (255, 255, 255, 0),
    spacing: int = 0
) -> Image.Image:
    """
    Merges two images either horizontally or vertically.
    direction: 'horizontal' (or 'h') | 'vertical' (or 'v')
    resize_mode:
        - 'fit': proportionally scales the second image to match the dimension of
the first image
                (height for horizontal, width for vertical)
        - 'pad': keeps original sizes and pads the smaller image with bg_color
    align:
        - for horizontal: 'top', 'center', 'bottom'
        - for vertical: 'left', 'center', 'right'
    spacing: optional blank gap between images in pixels
    """
    with Image.open(img1_path) as i1, Image.open(img2_path) as i2:
        img1 = i1.convert("RGBA")
        img2 = i2.convert("RGBA")
        dir_clean = direction.strip().lower()
        is_horiz = dir_clean in ("horizontal", "h", "horiz")

        w1, h1 = img1.size
        w2, h2 = img2.size
        print(f"    Image 1 '{os.path.basename(img1_path)}': {w1}x{h1}")
        print(f"    Image 2 '{os.path.basename(img2_path)}': {w2}x{h2}")

        if is_horiz:
            if resize_mode == "fit" and h1 != h2:
                # Proportionally scale img2 so its height matches img1

```

```

        target_h = h1
        target_w2 = max(1, int(round(w2 * (target_h / h2))))
        img2 = img2.resize((target_w2, target_h), Image.Resampling.LANCZOS)
        w2, h2 = img2.size
        print(f"    -> Rescaled Image 2 to {w2}x{h2} (matching height
{h1}px)")

    total_width = w1 + spacing + w2
    total_height = max(h1, h2)
    merged = Image.new("RGBA", (total_width, total_height), bg_color)

    # Calculate Y positions based on vertical alignment
    if align == "top":
        y1, y2 = 0, 0
    elif align == "bottom":
        y1, y2 = total_height - h1, total_height - h2
    else: # 'center'
        y1, y2 = (total_height - h1) // 2, (total_height - h2) // 2

    merged.paste(img1, (0, y1), mask=img1)
    merged.paste(img2, (w1 + spacing, y2), mask=img2)
    desc = f"Horizontal merge ({total_width}x{total_height})"

else: # Vertical
    if resize_mode == "fit" and w1 != w2:
        # Proportionally scale img2 so its width matches img1
        target_w = w1
        target_h2 = max(1, int(round(h2 * (target_w / w2))))
        img2 = img2.resize((target_w, target_h2), Image.Resampling.LANCZOS)
        w2, h2 = img2.size
        print(f"    -> Rescaled Image 2 to {w2}x{h2} (matching width
{w1}px)")

    total_width = max(w1, w2)
    total_height = h1 + spacing + h2
    merged = Image.new("RGBA", (total_width, total_height), bg_color)

    # Calculate X positions based on horizontal alignment
    if align == "left":
        x1, x2 = 0, 0
    elif align == "right":
        x1, x2 = total_width - w1, total_width - w2
    else: # 'center'

```

```

        x1, x2 = (total_width - w1) // 2, (total_width - w2) // 2

        merged.paste(img1, (x1, 0), mask=img1)
        merged.paste(img2, (x2, h1 + spacing), mask=img2)
        desc = f"Vertical merge ({total_width}x{total_height})"

        save_image(merged, output_path)
        print(f"    • {desc} completed successfully with mode='{resize_mode}',
align='{align}')"
        return merged

# TASK 2: WATERMARK CREATION (СТВОРЕННЯ ВОДЯНОГО ЗНАКУ)
def add_watermark(
    input_path: str,
    output_path: str,
    text: str = "CONFIDENTIAL",
    position: str | tuple[int, int] = "center",
    opacity: float = 0.5,
    font_size: int = 40,
    font_color: str | tuple = "white",
    angle: float = 0.0,
    stroke_width: int = 2,
    stroke_color: str | tuple = "black",
    shadow: bool = True,
    tile: bool = False,
    font_path: str | None = None,
    margin: int = 30
) -> Image.Image:
    """
    Applies a customizable watermark to an image.
    text: Watermark text string.
    position: 'center', 'top-left', 'top-right', 'bottom-left', 'bottom-right' or
    (x, y) coordinates.
    opacity: Transparency factor from 0.0 (transparent) to 1.0 (opaque) or 0..255.
    font_size: Font size in pixels.
    font_color: Watermark text color (name, hex, RGB).
    angle: Rotation angle in degrees (e.g., 30 or 45 for diagonal watermarks).
    stroke_width: Width of text outline border (preserves readability across
    contrast backgrounds).
    stroke_color: Outline color.
    shadow: When True, renders a subtle drop shadow to prevent blending with
    textures.

```



```

sw, sh = stamp.size

# Resolve coordinates
if isinstance(position, (tuple, list)) and len(position) >= 2:
    pos_x, pos_y = int(position[0]), int(position[1])
else:
    pos_str = str(position).strip().lower()
    if pos_str == "top-left":
        pos_x, pos_y = margin, margin
    elif pos_str == "top-right":
        pos_x, pos_y = max(0, bw - sw - margin), margin
    elif pos_str == "bottom-left":
        pos_x, pos_y = margin, max(0, bh - sh - margin)
    elif pos_str == "bottom-right":
        pos_x, pos_y = max(0, bw - sw - margin), max(0, bh - sh -
margin)

    else: # "center"
        pos_x, pos_y = max(0, (bw - sw) // 2), max(0, (bh - sh) // 2)

    watermark_layer.paste(stamp, (pos_x, pos_y), mask=stamp)
    print(f"    • Placed watermark '{text}' at ({pos_x}, {pos_y}),
size={font_size}px, "
        f"color={rgb_text}, opacity={alpha_val/255*100:.0f}%,
angle={angle}°")

# Composite base with watermark layer
result = Image.alpha_composite(base_rgba, watermark_layer)
save_image(result, output_path)

return result

def _create_watermark_stamp(
    text: str,
    font: ImageFont.FreeTypeFont | ImageFont.ImageFont,
    rgb_text: tuple[int, int, int],
    rgb_stroke: tuple[int, int, int],
    alpha: int,
    angle: float,
    stroke_width: int,
    shadow: bool
) -> Image.Image:
    """Renders a standalone rotated/shadowed watermark stamp image."""
    dummy_draw = ImageDraw.Draw(Image.new("RGBA", (1, 1)))
    bbox = dummy_draw.textbbox((0, 0), text, font=font, stroke_width=stroke_width)

```

```

text_w = bbox[2] - bbox[0]
text_h = bbox[3] - bbox[1]

pad = stroke_width * 2 + 10
stamp_w = text_w + pad * 2 + (4 if shadow else 0)
stamp_h = text_h + pad * 2 + (4 if shadow else 0)
stamp = Image.new("RGBA", (stamp_w, stamp_h), (0, 0, 0, 0))
draw = ImageDraw.Draw(stamp)

origin_x = pad - bbox[0]
origin_y = pad - bbox[1]

# Optional shadow for high contrast on noisy backgrounds
if shadow:
    shadow_alpha = int(alpha * 0.45)
    draw.text(
        (origin_x + 3, origin_y + 3),
        text,
        font=font,
        fill=(0, 0, 0, shadow_alpha),
        stroke_width=stroke_width,
        stroke_fill=(0, 0, 0, shadow_alpha)
    )

# Main text with outline stroke
text_fill = (rgb_text[0], rgb_text[1], rgb_text[2], alpha)
stroke_fill = (rgb_stroke[0], rgb_stroke[1], rgb_stroke[2], int(alpha * 0.8)) if
stroke_width > 0 else None

draw.text(
    (origin_x, origin_y),
    text,
    font=font,
    fill=text_fill,
    stroke_width=stroke_width,
    stroke_fill=stroke_fill
)

if angle != 0:
    stamp = stamp.rotate(angle, expand=True, resample=Image.Resampling.BICUBIC)

return stamp

```

```

# TASK 3: SLIDESHOW PLAYER & ANIMATED GIF EXPORT
def create_slideshow_gif(
    image_paths: Sequence[str],
    output_path: str,
    duration_sec: float = 1.5,
    loop: int = 0,
    target_size: tuple[int, int] | None = None
) -> str:
    """
    Creates an animated GIF slideshow from a sequence of images.
    Preserves aspect ratios by letterboxing onto a neat canvas.
    duration_sec: display duration per frame in seconds.
    loop: 0 means infinite loop.
    target_size: optional fixed frame size (width, height). Defaults to max image
    bounds.
    """

    valid_paths = [p for p in image_paths if os.path.isfile(p)]
    if not valid_paths:
        raise ValueError("No valid image files provided for slideshow GIF.")

    os.makedirs(os.path.dirname(output_path) or ".", exist_ok=True)

    # Determine canvas dimensions
    frames_orig = [Image.open(p) for p in valid_paths]
    if target_size is None:
        max_w = max(img.width for img in frames_orig)
        max_h = max(img.height for img in frames_orig)
        target_size = (max_w, max_h)
    tw, th = target_size

    processed_frames = []
    for img in frames_orig:
        canvas = Image.new("RGBA", (tw, th), (20, 24, 30, 255))
        # Proportionally fit image into (tw, th)
        ratio = min(tw / img.width, th / img.height)
        nw, nh = max(1, int(img.width * ratio)), max(1, int(img.height * ratio))
        fitted = img.convert("RGBA").resize((nw, nh), Image.Resampling.LANCZOS)
        px = (tw - nw) // 2
        py = (th - nh) // 2
        canvas.paste(fitted, (px, py), mask=fitted)
        processed_frames.append(canvas.convert("RGB"))

```

```

duration_ms = max(50, int(round(duration_sec * 1000)))
processed_frames[0].save(
    output_path,
    save_all=True,
    append_images=processed_frames[1:],
    duration=duration_ms,
    loop=loop,
    optimize=True
)
size_kb = get_size_kb(output_path)
print(f" • Slideshow GIF generated: {output_path} ({len(processed_frames)}
frames, "
      f"{tw}x{th}, delay={duration_sec:.2f}s, size={size_kb:.2f} KB)")
return output_path

def run_slideshow_cli(image_paths: Sequence[str], delay_sec: float = 1.0,
max_cycles: int = 1) -> None:
    """Non-GUI / headless console slideshow runner."""
    valid_paths = [p for p in image_paths if os.path.isfile(p)]
    if not valid_paths:
        print(" [!] No images found for slideshow.")
        return

    print(f"\n--- Running CLI Slideshow ({len(valid_paths)} images,
delay={delay_sec}s) ---")
    for cycle in range(max_cycles):
        for idx, p in enumerate(valid_paths, start=1):
            with Image.open(p) as img:
                print(f" [Slide {idx}/{len(valid_paths)}] Displaying:
{os.path.basename(p)} "
                      f"({img.width}x{img.height}, {img.mode})")
                time.sleep(delay_sec)
    print("--- CLI Slideshow complete ---\n")

def run_slideshow_gui(image_dir_or_files: str | Sequence[str], delay_sec: float =
2.0) -> None:
    """Launches the interactive Tkinter Slideshow Player GUI."""
    try:
        import tkinter as tk
        from tkinter import ttk
        from PIL import ImageTk
    except ImportError as e:

```

```

        print(f"Tkinter / ImageTk not available ({e}). Falling back to CLI mode.")
        if isinstance(image_dir_or_files, str) and
os.path.isdir(image_dir_or_files):
            files = sorted(glob.glob(os.path.join(image_dir_or_files, "*.*")))
        else:
            files = list(image_dir_or_files)
        run_slideshow_cli(files, delay_sec=delay_sec)
        return

# Collect images
if isinstance(image_dir_or_files, str) and os.path.isdir(image_dir_or_files):
    exts = ("*.png", "*.jpg", "*.jpeg", "*.bmp", "*.webp")
    files = []
    for ext in exts:
        files.extend(glob.glob(os.path.join(image_dir_or_files, ext)))
    files.sort()
elif isinstance(image_dir_or_files, str) and os.path.isfile(image_dir_or_files):
    files = [image_dir_or_files]
else:
    files = [f for f in image_dir_or_files if os.path.isfile(f)]

if not files:
    print("  [!] No valid images found to display.")
    return

root = tk.Tk()
root.title("Computer Graphics & Visualization – Slideshow Player (Lab #3)")
root.geometry("900x700")
root.minsize(600, 450)
root.configure(bg="#1E1E2E")

# State
current_idx = [0]
is_playing = [True]
current_delay = [int(delay_sec * 1000)]
timer_id = [None]
tk_img_ref = [None]

# UI Widgets
top_bar = tk.Frame(root, bg="#282A36", pady=8, padx=12)
top_bar.pack(fill=tk.X, side=tk.TOP)

title_label = tk.Label(

```

```

        top_bar,
        text="Laboratory Work #3 - Image Slideshow Player",
        font=("Segoe UI", 12, "bold"),
        fg="#F8F8F2",
        bg="#282A36"
    )
    title_label.pack(side=tk.LEFT)

    info_label = tk.Label(
        top_bar,
        text="",
        font=("Segoe UI", 10),
        fg="#8BE9FD",
        bg="#282A36"
    )
    info_label.pack(side=tk.RIGHT)

    # Canvas for display
    canvas = tk.Label(root, bg="#181825")
    canvas.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

    # Control toolbar at bottom
    control_frame = tk.Frame(root, bg="#282A36", pady=10, padx=15)
    control_frame.pack(fill=tk.X, side=tk.BOTTOM)

    def show_slide(idx: int):
        idx = idx % len(files)
        current_idx[0] = idx
        img_path = files[idx]

        cw = max(100, canvas.winfo_width() if canvas.winfo_width() > 1 else 860)
        ch = max(100, canvas.winfo_height() if canvas.winfo_height() > 1 else 520)

        with Image.open(img_path) as orig:
            ow, oh = orig.size
            ratio = min(cw / ow, ch / oh, 1.0)
            nw, nh = max(1, int(ow * ratio)), max(1, int(oh * ratio))
            display_img = orig.copy().resize((nw, nh), Image.Resampling.LANCZOS)
            photo = ImageTk.PhotoImage(display_img)
            tk_img_ref[0] = photo
            canvas.configure(image=photo)

        filename = os.path.basename(img_path)

```

```

        info_label.config(text=f"Slide {idx + 1} / {len(files)} | {filename}
({ow}x{oh} px)")

def next_slide():
    show_slide(current_idx[0] + 1)

def prev_slide():
    show_slide(current_idx[0] - 1)

def toggle_play():
    is_playing[0] = not is_playing[0]
    btn_play.config(text="⏸ Pause" if is_playing[0] else "▶ Play")
    if is_playing[0]:
        schedule_next()
    elif timer_id[0]:
        root.after_cancel(timer_id[0])
        timer_id[0] = None

def schedule_next():
    if timer_id[0]:
        root.after_cancel(timer_id[0])
    if is_playing[0]:
        timer_id[0] = root.after(current_delay[0], auto_advance)

def auto_advance():
    if is_playing[0]:
        next_slide()
        schedule_next()

def update_delay(val):
    current_delay[0] = max(200, int(float(val) * 1000))
    delay_label.config(text=f"Speed: {float(val):.1f}s")
    if is_playing[0]:
        schedule_next()

# Controls
btn_prev = tk.Button(control_frame, text="⏮ Prev", command=prev_slide,
                      font=("Segoe UI", 9, "bold"), bg="#6272A4", fg="white",
                      padx=10, relief=tk.FLAT)
btn_prev.pack(side=tk.LEFT, padx=5)

btn_play = tk.Button(control_frame, text="⏸ Pause", command=toggle_play,
                     font=("Segoe UI", 9, "bold"), bg="#50FA7B", fg="#282A36",
                     padx=15, relief=tk.FLAT)

```

```

    btn_play.pack(side=tk.LEFT, padx=5)

    btn_next = tk.Button(control_frame, text="Next ►", command=next_slide,
                          font=("Segoe UI", 9, "bold"), bg="#6272A4", fg="white",
                          padx=10, relief=tk.FLAT)
    btn_next.pack(side=tk.LEFT, padx=5)

    delay_label = tk.Label(control_frame, text=f"Speed: {delay_sec:.1f}s",
                           font=("Segoe UI", 9), fg="#F8F8F2", bg="#282A36",
                           padx=10)
    delay_label.pack(side=tk.LEFT, padx=5)

    slider = tk.Scale(control_frame, from_=0.5, to=5.0, resolution=0.1,
                      orient=tk.HORIZONTAL,
                      command=update_delay, bg="#282A36", fg="#F8F8F2",
                      highlightthickness=0,
                      length=150, showvalue=False)
    slider.set(delay_sec)
    slider.pack(side=tk.LEFT, padx=5)

    btn_close = tk.Button(control_frame, text="Close ✕", command=root.destroy,
                          font=("Segoe UI", 9), bg="#FF5555", fg="white", padx=10,
                          relief=tk.FLAT)
    btn_close.pack(side=tk.RIGHT, padx=5)

    # Keyboard shortcuts
    root.bind("<Left>", lambda e: prev_slide())
    root.bind("<Right>", lambda e: next_slide())
    root.bind("<space>", lambda e: toggle_play())
    root.bind("<Escape>", lambda e: root.destroy())
    root.bind("<Configure>", lambda e: show_slide(current_idx[0]))

    root.update()
    show_slide(0)
    schedule_next()
    root.mainloop()

# LAB 1 & LAB 2 FOUNDATIONAL FUNCTIONS
...

# INTERACTIVE CLI MENU
def main() -> None:
    while True:
        print("\n" + "=" * 65)

```

```

    print("    LABORATORY WORK #3: IMAGE PROCESSING (MERGE, WATERMARK,
SLIDESHOW)")

    print("=" * 65)

    print("--= LAB 1: FORMATS & RESIZING ==")

    print(" 1. Convert image formats (JPG, PNG, BMP, WEBP)")
    print(" 2. Resize images (proportional scaling)")

    print("--= LAB 2: TRANSPARENCY, CROPPING & CONTRAST ==")

    print(" 3. Change transparency (putalpha)")
    print(" 4. Remove background color (Chroma Key)")
    print(" 5. Overlay semi-transparent layer")
    print(" 6. Split image into N parts")
    print(" 7. Crop region (keep or remove)")
    print(" 8. Adjust contrast (NewY formula)")

    print("--= LAB 3: MERGE, WATERMARK & SLIDESHOW ==")

    print(" 9. Merge two images (Horizontal / Vertical)")
    print("10. Add custom watermark (text, position, opacity, color, angle)")
    print("11. Add tiled watermark pattern (security grid)")
    print("12. Launch interactive Slideshow Player (GUI Tkinter)")
    print("13. Export images to animated Slideshow GIF")
    print("14. Exit")

    print("-" * 65)

    try:
        choice = input("Select an option (1-15): ").strip()
    except (EOFError, KeyboardInterrupt):
        print("\nExiting program.")
        break

    # --- LAB 1 ---

    if choice == "1":
        inp = input("Enter input image path(s) (comma-separated): ").strip()
        paths = [p.strip() for p in inp.split(",") if p.strip()]
        fmts = input("Enter target formats (e.g. png, jpg, bmp, webp):
").strip()

        fmt_list = [f.strip() for f in fmts.split(",") if f.strip()]
        out_dir = input("Enter output directory [default 'output_format']:
").strip() or "output_format"

        convert_formats(paths, fmt_list, out_dir)

    elif choice == "2":
        inp = input("Enter input image path(s) (comma-separated): ").strip()
        paths = [p.strip() for p in inp.split(",") if p.strip()]
        out_dir = input("Enter output directory [default 'output_resized']:
").strip() or "output_resized"

```

```

w_str = input("Enter target width (or press Enter to skip): ").strip()
h_str = input("Enter target height (or press Enter to skip): ").strip()
tw = int(w_str) if w_str else None
th = int(h_str) if h_str else None
resize_images(paths, out_dir, target_w=tw, target_h=th)

# --- LAB 2 ---
elif choice == "3":
    inp = input("Enter input image path: ").strip()
    out = input("Enter output image path: ").strip()
    val = int(input("Enter alpha value (0-255): ").strip())
    set_transparency(inp, out, val)

elif choice == "4":
    inp = input("Enter input image path: ").strip()
    out = input("Enter output image path: ").strip()
    make_color_transparent(inp, out)

elif choice == "5":
    base = input("Enter base image path: ").strip()
    over = input("Enter overlay image path: ").strip()
    out = input("Enter output image path: ").strip()
    pos_x = int(input("Enter X coordinate for overlay: ").strip())
    pos_y = int(input("Enter Y coordinate for overlay: ").strip())
    opac = float(input("Enter overlay opacity (0.0..1.0): ").strip())
    overlay_layer(base, over, out, pos=(pos_x, pos_y), opacity=opac)

elif choice == "6":
    inp = input("Enter input image path: ").strip()
    out_dir = input("Enter output directory for parts: ").strip()
    n = int(input("Enter number of parts: ").strip())
    split_image(inp, out_dir, num_parts=n)

elif choice == "7":
    inp = input("Enter input image path: ").strip()
    out = input("Enter output image path: ").strip()
    box_str = input("Enter coordinates X1,Y1,X2,Y2 (comma-separated): ").strip()
    box = tuple(map(int, box_str.split(",")))
    sub = input("Keep region (1) or Remove region (2): ").strip()
    if sub == "1":
        crop_keep_region(inp, out, box)
    else:

```

```

        crop_remove_region(inp, out, box)

    elif choice == "8":
        inp = input("Enter input image path: ").strip()
        out = input("Enter output image path: ").strip()
        k = float(input("Enter contrast factor K (>1 increase, <1 decrease): ").strip())
        adjust_contrast(inp, out, k)

    # --- LAB 3 ---
    elif choice == "9":
        i1 = input("Enter path to first image: ").strip()
        i2 = input("Enter path to second image: ").strip()
        out = input("Enter output path (e.g., output_merge/merged.png): ").strip()
        direction = input("Enter direction (h - horizontal, v - vertical) [default 'h']: ").strip() or "h"
        mode = input("Enter resize mode (fit - rescale, pad - pad canvas) [default 'fit']: ").strip() or "fit"
        merge_images(i1, i2, out, direction=direction, resize_mode=mode)

    elif choice == "10":
        inp = input("Enter input image path: ").strip()
        out = input("Enter output image path: ").strip()
        txt = input("Enter watermark text: ").strip() or "WATERMARK"
        pos = input("Enter position ('center', 'top-left', 'bottom-right' or 'X,Y'): ").strip() or "center"
        if "," in pos:
            pos = tuple(map(int, pos.split(",")))
        opac = float(input("Enter opacity (0.0 .. 1.0 or 0 .. 255) [default 0.5]: ").strip() or "0.5")
        fsize = int(input("Enter font size in px [default 40]: ").strip() or "40")
        color = input("Enter color (name, hex or RGB) [default 'white']: ").strip() or "white"
        angle = float(input("Enter rotation angle in degrees [default 0]: ").strip() or "0")
        add_watermark(inp, out, text=txt, position=pos, opacity=opac, font_size=fsize,
                      font_color=color, angle=angle)

    elif choice == "11":
        inp = input("Enter input image path: ").strip()
        out = input("Enter output image path: ").strip()
        txt = input("Enter pattern text: ").strip() or "CONFIDENTIAL"

```

```

        opac = float(input("Enter opacity [default 0.3]: ").strip() or "0.3")
        add_watermark(inp, out, text=txt, opacity=opac, tile=True)

    elif choice == "12":
        folder = input("Enter folder path with images [default 'output_merge']:"
            ).strip() or "output_merge"
        delay = float(input("Enter slide delay in seconds [default 2.0]:
            ").strip() or "2.0")
        run_slideshow_gui(folder, delay_sec=delay)

    elif choice == "13":
        folder = input("Enter folder path with images [default
            'output_watermark']: ").strip() or "output_watermark"
        out = input("Enter output GIF path [default
            'output_slideshow/slideshow.gif']: ").strip() or "output_slideshow/slideshow.gif"
        delay = float(input("Enter frame duration in seconds [default 1.5]:
            ").strip() or "1.5")
        exts = ("*.png", "*.jpg", "*.jpeg")
        files = []
        for ext in exts:
            files.extend(glob.glob(os.path.join(folder, ext)))
        files.sort()
        if files:
            create_slideshow_gif(files, out, duration_sec=delay)
        else:
            print(f"No image files found in '{folder}'")

    elif choice == "14":
        print("\nProgram finished!")
        break

if __name__ == "__main__":
    main()

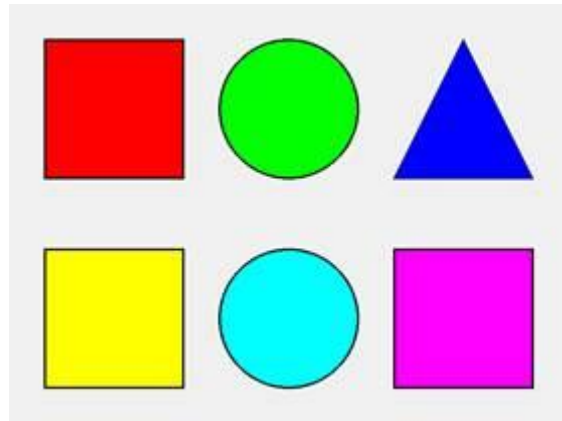
```

РЕЗУЛЬТАТИ ВИКОНАННЯ ПРОГРАМИ

Вхідні тестові матеріали



test_landscape.jpg (1200x800)



test_shapes.png (800x600)

Результати Завдання 1: Горизонтальне та вертикальне об'єднання

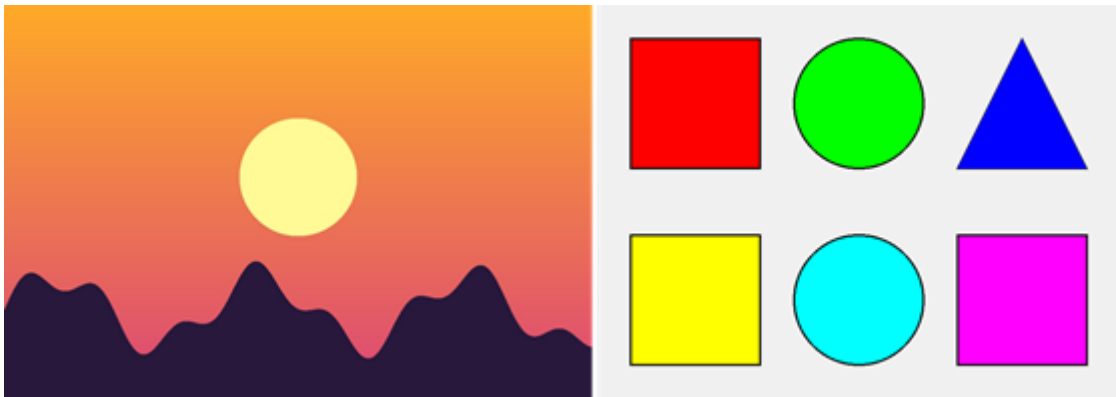


Рис. 1. Горизонтальне об'єднання у режимі 'fit': масштабування за висотою (2277x800 px, 170.12 KB).



Рис. 2. Горизонтальне об'єднання у режимі 'pad': збереження розмірів та центральне вирівнювання (2020x800 px, 119.75 KB).

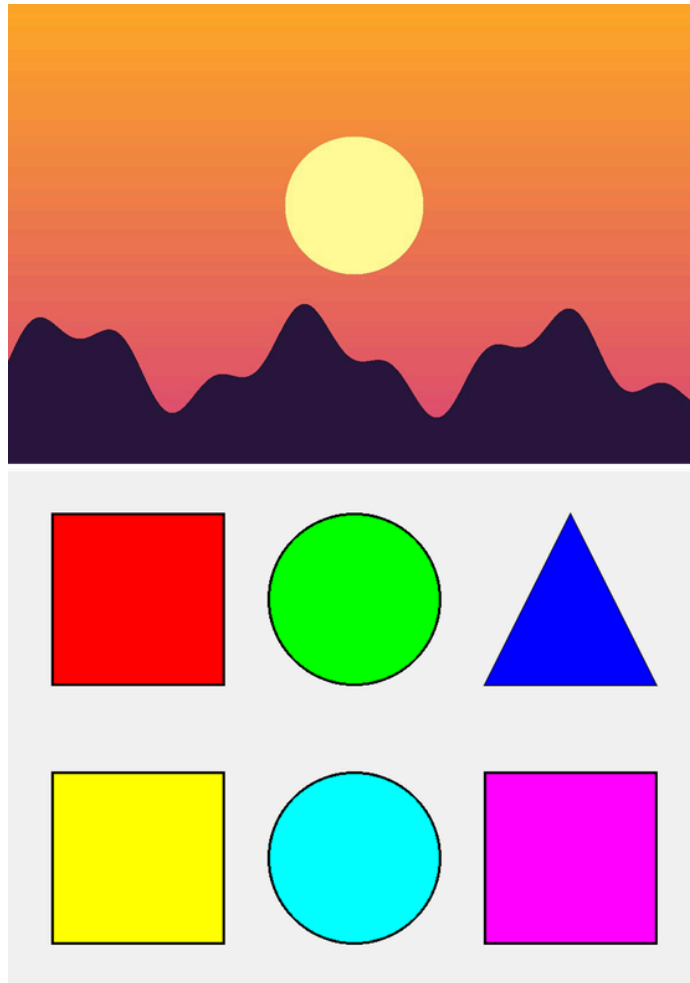


Рис. 3. Вертикальне об'єднання у режимі 'fit': масштабування за шириною (1200x1710 px, 162.63 KB).

Результати Завдання 2: Нанесення захисних водяних знаків



Рис. 4. Діагональний водяний знак по центру (кут 35°, розмір 56px, opacity 60%, з контуром та тінню).



Рис. 5. Кутівий водяний знак 'IPZ-23 • Lab #3' у правому нижньому куті (opacity 75%, темно-синій контур).

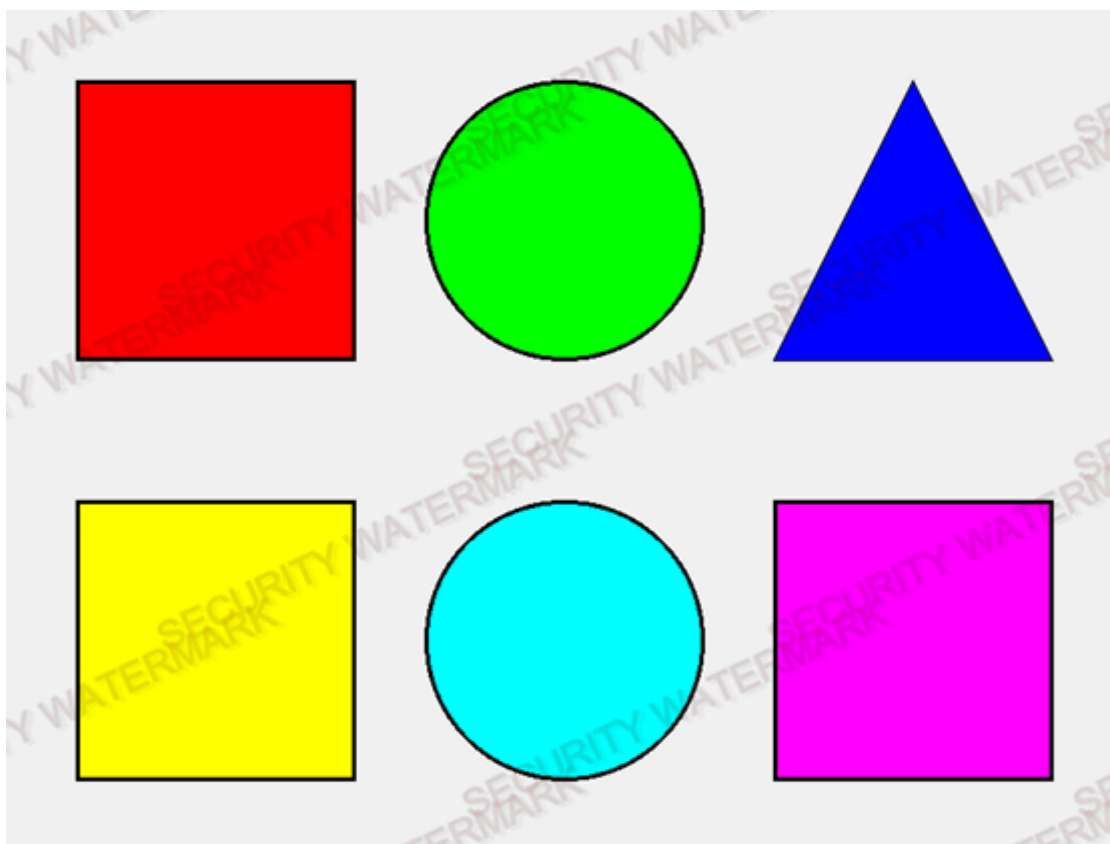


Рис. 6. Захисна сітка діагональних водяних знаків (Tiling Mode, кут 25°, alpha=89/255).

Результати Завдання 3: Інтерактивний програвач слайд-шоу (Tkinter) та експорт GIF

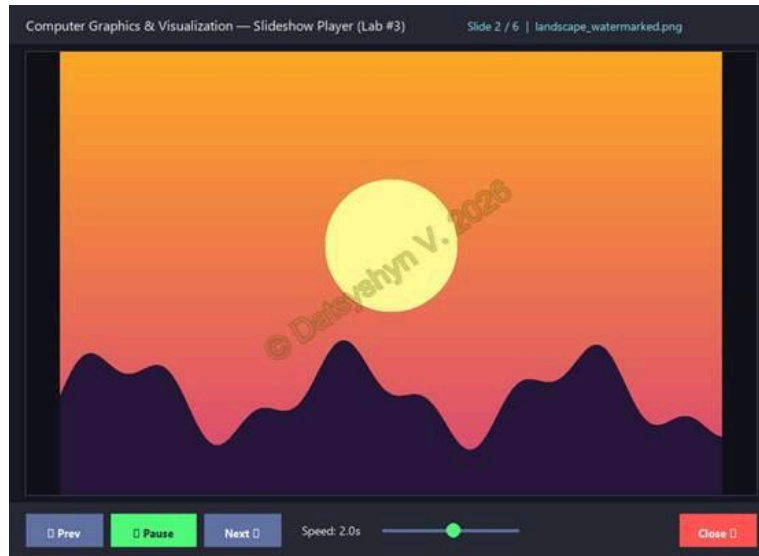


Рис. 7. Інтерфейс розробленого програвача SlideshowPlayer (Tkinter GUI): панель навігації та повзунок швидкості.

ВИСНОВКИ

Під час виконання лабораторної роботи № 3 було успішно досягнуто поставленої мети: поглиблено знання методів цифрової обробки зображень та реалізовано прикладне програмне забезпечення для просторового об'єднання растрових даних, нанесення адаптивних водяних знаків та автоматизованого відтворення серії кадрів у форматі слайд-шоу.

У процесі виконання роботи отримано такі теоретичні та практичні результати:

1. Досліджено та реалізовано алгоритми просторового об'єднання графічних матеріалів: створено механізми горизонтальної та вертикальної конкатенації, розв'язано проблему геометричної колізії кадрів різної роздільної здатності шляхом впровадження пропорційного ресемплінгу Lanczos (режим 'fit') та розширення канви із центрованим розташуванням (режим 'pad').

2. Розроблено багаторівневий модуль нанесення захисних водяних знаків із підтримкою альфа-каналу: доведено високу ефективність використання контурної обводки (stroke) та зміщеної тіні (shadow), які запобігають візуальному злиттю тексту зі світлими або темними ділянками фону. Реалізовано довільну просторову орієнтацію тексту під кутом та захисний патерн суцільного тайлінгу (tiling) проти кадрування.

3. Створено повноцінний графічний додаток слайд-шоу на базі бібліотеки Tkinter з адаптивним масштабуванням зображень під розмір вікна, інтерактивним керуванням швидкістю, перемиканням кадрів та автоматизованим експортом в анімований формат GIF.

4. Забезпечено спадковість кодової бази лабораторних робіт № 1 та № 2, що підтверджує модульність та масштабованість розробленої архітектури.

ВІДПОВІДІ НА ПИТАННЯ ДЛЯ САМОКОНТРОЛЮ

1. Що таке об'єднання зображень, і які основні методи для цього існують?

Об'єднання зображень — це процес формування єдиного результуючого растрового кадру з двох або більше вихідних зображень. Основними методами є:

- Просторове зшивання (Concatenation / Stacking): розміщення зображень пліч-о-пліч за горизонталлю або вертикаллю;
- Альфа-змішування (Alpha Blending): накладання шарів з прозорістю за формулою $C = C1 \cdot \alpha + C2 \cdot (1 - \alpha)$;
- Маскове поєднання: об'єднання частин зображень через бінарну чи градієнтну маску;
- Панорамне зшивання (Stitching): вирівнювання та поєднання з перекриттям на основі пошуку ключових точок (SIFT/ORB) та усунення швів.

2. Як програмно реалізується об'єднання зображень горизонтально та вертикально? Які фактори потрібно враховувати при виборі напрямку об'єднання?

Програмно створюється нове порожнє полотно `Image.new()` розмірами:

- Для горизонтального: $Width = W1 + W2 + spacing$, $Height = \max(H1, H2)$;
- Для вертикального: $Width = \max(W1, W2)$, $Height = H1 + H2 + spacing$.

Після цього за допомогою методу `paste()` вставляються перше зображення у позицію (0, 0), а друге — у ($W1 + spacing$, 0) для горизонтального або (0, $H1 + spacing$) для вертикального об'єднання.

При виборі напрямку враховують співвідношення сторін вихідних зображень, цільовий контекст (наприклад, альбомна або книжкова орієнтація екрана/документа) та сюжетний зміст композиції (порівняння «до/після» зазвичай роблять горизонтальним, а вертикальні стрічки — для мобільних інтерфейсів).

3. Як змінюється розмір кінцевого зображення після його об'єднання з іншими?

Фізична роздільна здатність (кількість пікселів) збільшується адитивно:

- За горизонталлю: сумарна ширина дорівнює сумі ширин ($W1 + W2$), а висота визначається більшим із кадрових розмірів $\max(H1, H2)$;
- За вертикаллю: сумарна висота дорівнює сумі висот ($H1 + H2$), а ширина — $\max(W1, W2)$.

Якщо застосовується пропорційне підлаштування ('fit'), розміри другого кадру змінюються за коефіцієнтом масштабування. Обсяг файлу на диску також зростає пропорційно площі нового полотна та складності стиснення пікселів.

4. Що таке водяний знак, і для чого його використовують у зображеннях?

Водяний знак (Watermark) — це напівпрозорий текстовий або графічний відбиток (логотип, напис, ідентифікатор), накладений поверх основного зображення. Його використовують для:

- 1) Захисту авторських прав та інтелектуальної власності від несанкціонованого копіювання;
- 2) Брендування та підвищення впізнаваності автора або організації;

- 3) Вказівки статусу документа (наприклад, «DRAFT», «CONFIDENTIAL», «SAMPLE»);
- 4) Монетизації контенту на фотостоках (попередній перегляд перед покупкою).

5. Які параметри водяного знаку можна змінювати, і як це впливає на вигляд зображення?

До основних регульованих параметрів належать:

- Рівень прозорості (Opacity / Alpha): балансує між захистом і непомітністю для сприйняття основного контенту;
- Розмір шрифту (Font Size): занадто великий перекриває деталі, занадто дрібний є нечитабельним;
- Колір (Color): контрастний до тла виділяє марку, нейтральний робить її стриманою;
- Позиція (Position): кути забезпечують естетику, а центр — максимальний захист від обрізання;
- Кут нахилу (Angle): діагональне розташування ускладнює видалення та перетинає ключові об'єкти сцени;
- Обводка (Stroke) та тінь (Shadow): гарантують чіткість тексту на будь-якому тлі.

7. Як вибір прозорості водяного знаку впливає на сприйняття зображення?

Низька прозорість (висока непрозорість, $\text{Alpha} > 80\%$) робить знак домінуючим, відволікає увагу глядача від основного змісту та руйнує естетичну цінність кадру. Надмірно висока прозорість ($\text{Alpha} < 15\%$) робить знак майже непомітним і легким для видалення фільтрами чи стисненням. Оптимальним є діапазон 30%–60%, де знак надійно фіксує авторство, не заважаючи спогляданню композиції.

8. Яким чином можна налаштувати позицію водяного знаку на зображенні (наприклад, верхній лівий кут, центр)?

Позиція налаштовується розрахунком координат (X, Y) відносно розмірів зображення (W, H) та габаритів самого водяного знаку (w_mark, h_mark):

- Верхній лівий кут: $X = \text{margin}$, $Y = \text{margin}$;
- Верхній правий кут: $X = W - w_mark - \text{margin}$, $Y = \text{margin}$;
- Нижній лівий кут: $X = \text{margin}$, $Y = H - h_mark - \text{margin}$;
- Нижній правий кут: $X = W - w_mark - \text{margin}$, $Y = H - h_mark - \text{margin}$;
- Центр: $X = (W - w_mark) / 2$, $Y = (H - h_mark) / 2$.

9. Які методи можна застосувати для збереження водяного знаку, щоб він не зливався із зображенням?

Для запобігання візуальному злиттю застосовують такі інженерні методи:

- 1) Контурна обводка (Outline Stroke): додавання тонкої контрастної облямівки іншого кольору (наприклад, чорний контур навколо білого тексту);
- 2) Відкидання тіні (Drop Shadow): рендеринг зміщеної темної тіні створює псевдо-3D ефект і відокремлює символи від фону;
- 3) Напівпрозора підкладка (Bounding Box Bkg): накладання тексту на прямокутну напівпрозору плашку;
- 4) Інверсне / адаптивне накладання (Difference / Luminance-based): зміна кольору шрифту залежно від локальної яскравості підкладки.

10. Як можна забезпечити певну затримку між відображенням зображень у слайд-шоу?

У графічних інтерфейсах (Tkinter) використовують асинхронні неблокуючі таймери подій: метод `root.after(ms, callback)`, який реєструє функцію зміни кадру в черзі подій головного циклу, не заморожуючи інтерфейс користувача.

У консольних скриптах або потоках використовують функцію `time.sleep(delay_sec)`.

В анімованих форматах файлів (GIF, WebP) тривалість затримки записується безпосередньо у метадані заголовка кожного кадру (параметр `duration` у мілісекундах).

11. Які формати зображень найкраще підходять для створення слайд-шоу?

Вибір формату залежить від каналу розповсюдження:

- Для автономних анімованих файлів: GIF (має 100% сумісність з усіма переглядачами, але обмежений 256 кольорами) та WebP / APNG (сучасні формати з 24-бітним повноколірним відображенням та високим ступенем стиснення);
- Для програвачів локальних програм (GUI / Веб): JPEG (для високої роздільної здатності фотографій при малому розмірі) та PNG (для графіки, що потребує ідеальної чіткості та прозорості);
- Для відеоформатів: MP4 (H.264/H.265), що забезпечує апаратне прискорення та плавні кінематографічні переходи.